

Proposta Técnica: Correção de Direção e Controle de Motores (UD-H1)

Este documento detalha o diagnóstico técnico e as alterações de código propostas para resolver o problema do robô **UD-H1** puxar para o lado, derrapar (*skidding*) ou oscilar ao andar, garantindo que ele siga as direções comandadas de forma precisa e linear.

1. Diagnóstico do Problema (O que estava acontecendo?)

A equipe identificou que o robô apresentava dificuldades em manter uma trajetória retilínea estável. A nossa análise do firmware do Arduino e do driver do ROS 2 revelou **quatro causas principais** para essa instabilidade:

A. Frequência de Controle Muito Baixa (10 Hz)

A malha de controle no Arduino rodava a cada **100ms**. Em motores de hoverboard (que possuem torque e velocidade elevados), essa latência de 0,1s é excessiva. O robô desviava fisicamente antes que o Arduino pudesse ler o erro e ajustar a potência, provocando guinadas e derrapagens.

B. Controle Apenas Integral (I-only)

A fórmula do controle de velocidade era calculada acumulando o erro direto no PWM (`pwm += erro * kp`). Matematicamente, isso se comporta apenas como um termo **Integral**. Controladores puramente integrais têm resposta lenta para iniciar o movimento (rampa lenta) e sofrem de *overshoot* (passam do valor alvo e ficam oscilando em "S").

C. Conflito entre Malha Fechada e TRIM Estático

A equipe havia adicionado um TRIM estático (`TRIM_DIR = -0.07`) para tentar corrigir a diferença física de velocidade entre os motores: sem o TRIM, um motor girava a **192 RPM** e o outro a **216 RPM** sob o mesmo comando. Como o robô está em malha fechada, se o TRIM força a roda a girar mais devagar, o controlador lê a velocidade menor e aumenta o PWM para compensar. Isso gerava uma disputa constante entre a atenuação do TRIM (que não estava bem ajustado) e a correção da malha fechada, causando instabilidade.

D. Comportamento Anômalo de Sinais no ROS 2 (Nav2 & Gazebo)

Havia uma negação de eixos na odometria ($v_{linear} = -((v_{dir} + v_{esq})/2.0)$). A equipe observou que: * Durante a **teleoperação manual**, o sentido de movimentação do robô no RViz2 estava correto. * Contudo, o problema ocorria ao ativar a navegação autônoma pelo **Nav2**: o mapa quebrava e o RViz2 ficava completamente instável ("bugado") ao tentar alcançar o ponto marcado. * No ambiente simulado do **Gazebo**, o robô exibia um comportamento de "teletransporte".

2. Escopo e Priorização (Alinhamento da Equipe)

Conforme acordado com a equipe, as prioridades foram divididas da seguinte forma: * **Foco Imediato (Fase 2 - Atual)**: Resolver a derrapada (*skidding*) do robô e garantir que ele ande em linha reta de forma consistente no ambiente real por meio do novo controle PI + Feedforward e das interrupções nativas de hardware do Arduino. * **Postergado (Pós-dia 13)**: A resolução detalhada dos problemas de navegação autônoma no Nav2 (quebra do mapa no RViz2) e o comportamento de teletransporte no simulador Gazebo.

3. Alterações Propostas e Implementadas

As seguintes melhorias foram aplicadas no repositório local:

A. Novo Firmware do Arduino

- **Controle a 50 Hz (20ms)**: Reduzida a latência de controle em 5 vezes, permitindo correções rápidas de trajetória.
- **Controlador PI + Feedforward**: Implementada a fórmula completa de controle Proporcional-Integral mais um ganho de Feedforward ($FeedForward = 100.0$). Isso faz com que os motores respondam instantaneamente ao comando de velocidade alvo (Feedforward), cabendo ao PI apenas a correção de pequenas perturbações do solo.
- **Interrupções de Hardware Nativas**: O código foi alterado para usar `attachInterrupt` direto de hardware nos pinos 2 e 3 (`INT0` e `INT1`) em vez da biblioteca `PinChangeInterrupt`, o que elimina a possibilidade de perda de pulsos do encoder em velocidades altas.
- **Zerar Integradores**: Integradores de erro são reiniciados ao parar o robô, prevenindo acúmulo de potência residual.
- **Remoção do TRIM**: TRIM estático zerado, permitindo que a malha de controle controle a linearidade.

B. Novo Driver ROS 2

- **Rampas de Aceleração Suaves:** Implementamos limites dinâmicos de aceleração linear (0.8 m/s^2) e angular (1.5 rad/s^2). Mesmo que receba comandos bruscos do teclado ou planejador, o robô acelerará de forma suave, evitando derrapagens por perda de tração.
- **Remoção de Negações:** Odometria linear e angular agora reportam valores estritamente positivos ao andar para a frente e girar para a esquerda (anti-horário), em conformidade total com o padrão ROS.

C. Utilitário Customizado WASD

Criamos o nó `wasd_teleop` para substituir o teclado padrão do ROS. A equipe agora controla o robô usando a convenção de jogos padrão no terminal: * `W` / `S` : Ir para frente / trás (velocidade segura de 0.2 m/s) * `A` / `D` : Girar sobre o próprio eixo para a esquerda / direita (0.6 rad/s) * `Espaço` : Parar o robô instantaneamente.

4. Por que usar os Parâmetros de Controle (Placeholders) em vez de TRIM Estático?

Substituímos o uso do TRIM estático (`TRIM_DIR = -0.07`) na saída do motor pelas variáveis e constantes de controle dinâmico (representadas originalmente pelos placeholders de código no documento, como `FeedForward`, `kp` e `ki`). Essa decisão baseia-se em princípios fundamentais de controle:

1. Compensação Dinâmica vs. Estática:

2. O **TRIM estático** aplica uma atenuação de potência fixa e cega. Ele não se adapta a variações de voltagem da bateria, rugosidade do solo, inclinação do terreno ou desgaste mecânico dos motores ao longo do tempo.

Os **Parâmetros de Malha Fechada** (`kp`, `ki` e `FeedForward`) recalculam continuamente a tensão necessária a cada ciclo de 20ms baseando-se no erro real de velocidade (diferença entre o alvo e o lido pelo encoder).

Prevenção do Conflito de Controle ("Cabo de Guerra"):

5. Quando o TRIM estático reduz a potência de uma roda, o robô gira mais devagar daquele lado. O controlador PI lê essa velocidade abaixo do setpoint, identifica isso como um erro e aumenta a potência para compensar.

Isso gera uma disputa cíclica na qual o TRIM tenta atenuar e a malha fechada tenta acelerar, causando oscilações bruscas (movimento em "S") e derrapagens.

Partida Simétrica Ajustada por Feedforward:

8. Para corrigir a assimetria na partida física (quando um motor vence a inércia mais rápido que o outro, como a variação de 192 RPM vs 216 RPM relatada), o correto é calibrar as constantes de ganho preditivo de partida individualmente para cada motor (`feedForwardEsq` e `feedForwardDir`). Isso atua de maneira limpa antes da ação de correção do PI, ao invés de limitar o PWM final.

5. Roteiro de Teste Proposto para a Equipe

Para validar as alterações fisicamente, a equipe deve seguir estes 3 passos simples no laboratório:

Passo 1: Teste com Rodas Suspensas (Estático)

1. Colocar o robô em cima de um suporte para que as rodas girem livremente no ar.
2. Compilar o pacote e rodar a teleoperação: `bash colcon build --packages-select serial_com_py source install/setup.bash ros2 run serial_com_py wasd_teleop`
3. Pressionar **W** e verificar se as duas rodas giram na mesma velocidade para a frente. Pressionar **S** e verificar se giram para trás.
4. Pressionar **A** e verificar se a roda esquerda gira para trás e a direita gira para frente (sentido de rotação para a esquerda).

Passo 2: Teste de Consistência de Sentido no RViz2

1. Abrir a visualização do robô (`rviz2`).
2. Empurrar o robô fisicamente para frente e verificar se a odometria e o laser se deslocam para frente na tela.
3. Rotacionar o robô fisicamente para a esquerda e verificar se a seta vermelha de odometria gira para a esquerda.

Passo 3: Teste de Pista de 1 Metro (Dinâmico)

1. Posicionar o robô no chão, alinhado com uma fita métrica.
2. Comandar o robô para avançar a `0.15 m/s` (segurando a tecla **W**).
3. O robô deve andar exatamente 1 metro em linha reta. O desvio lateral medido em relação à linha guia deve ser **menor do que 5 cm**.